HookLens API 명세서

이 문서의 응답 예시는 로컬에서 운영 구성(Nginx + 백엔드 + MySQL)을 띄우고 실제로 호출해서 받은 값입니다. 토큰과 slug만 예시 값으로 바꿨습니다.

GITHUB https://github.com/songjeongseon1/hooklens

- 기준 버전: 1차 MVP (P0 요구사항 8개)
- Base URL: `http://{host}` (로컬 개발은 Vite 프록시, 운영은 Nginx가 `/api`, `/h`를 백엔드로 넘김)
- 문자셋: UTF-8, 시간: ISO-8601 + KST 오프셋 (`2026-10-24T15:11:36.107941+09:00`)

목차

1. 공통 규칙
2. API 목록
3. 엔드포인트 발급
4. 웹훅 수신
5. 요청 목록 조회
6. 요청 상세 조회
7. 새 요청 실시간 구독 (SSE)
8. 응답 설정 조회
9. 응답 설정 저장
10. 에러 코드

1. 공통 규칙

응답 형식

관리 API(`/api/**`)는 성공과 실패 모두 같은 모양으로 응답합니다.

```
{ "success": true,  "data": { ... }, "error": null }
{ "success": false, "data": null,    "error": { "code": "UNAUTHORIZED", "message": "인증이 필요합니다." } }
```

웹훅 수신(`/h/**`)은 사용자가 설정한 응답을 그대로 돌려주기 때문에 이 형식을 쓰지 않습니다. 단, 주소가 없거나(404) 만료된 경우(410)는 위 실패 형식으로 응답합니다.

인증

엔드포인트를 발급하면 응답에 `anonymousToken`이 **한 번만** 들어 있습니다. 서버에는 SHA-256 해시만 저장하므로 다시 조회할 수 없습니다. 이후 그 엔드포인트의 관리 API는 모두 헤더로 토큰을 보냅니다.

```
X-Anonymous-Token: wklXhifFLxoLJQcF_LQdBmMckrYMe3zWGhNnwxsOYQA
```

인증이 필요한 API는 아래 순서로 검사합니다.

순서	조건	응답
1	`X-Anonymous-Token` 헤더 없음 (또는 빈 값)	401 `UNAUTHORIZED`
2	slug에 해당하는 엔드포인트가 없음	404 `ENDPOINT_NOT_FOUND`
3	만료됨 (발급 후 24시간)	410 `ENDPOINT_EXPIRED`
4	토큰이 틀림	403 `FORBIDDEN_ENDPOINT`

토큰 유무를 가장 먼저 보는 이유는 DB 조회 없이 할 수 있는 확인이기 때문입니다. 토큰 비교는 `MessageDigest.isEqual`로 해서 비교 시간으로 토큰을 추측할 수 없게 했습니다.

입력 검증

검증에 실패하면 400 `INVALID_INPUT`이고, `message`에 어느 값이 왜 틀렸는지 들어갑니다. 여러 개가 틀리면 `, `로 이어 붙입니다.

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "INVALID_INPUT",
    "message": "statusCode는 100~599 사이여야 합니다., delayMs는 0~5000 사이여야 합니다."
  }
}
```

2. API 목록

메서드	경로	설명	인증	요구사항
POST	`/api/endpoints`	엔드포인트 발급	없음	REQ-EP-001
ANY	`/h/{slug}/**`	웹훅 수신	없음	REQ-HOOK-001·002, EP-002
GET	`/api/endpoints/{slug}/requests`	요청 목록 (최신순, 커서)	토큰	REQ-LOG-002
GET	`/api/endpoints/{slug}/requests/{requestId}`	요청 상세	토큰	REQ-LOG-003
GET	`/api/endpoints/{slug}/stream`	새 요청 실시간 구독 (SSE)	토큰	REQ-LOG-001
GET	`/api/endpoints/{slug}/response`	응답 설정 조회	토큰	REQ-RES-001
PUT	`/api/endpoints/{slug}/response`	응답 설정 저장	토큰	REQ-RES-001

3. 엔드포인트 발급

```
POST /api/endpoints
```

가입 없이 웹훅을 받을 주소를 만듭니다. 바디 없이 보내도 됩니다.

요청 바디 (선택)

필드	타입	필수	설명
name	string	X	엔드포인트 이름, 50자 이하. 비어 있거나 공백만 있으면 이름 없음(null)

```
{ "name": "토스 결제 웹훅 테스트" }
```

응답 `201 Created`

필드	타입	설명
slug	string	영숫자 10자
url	string	외부 서비스에 등록할 웹훅 주소
name	string | null	이름
anonymousToken	string	관리 API용 토큰. **이 응답에서만 제공**
expiresAt	string	만료 시각 (발급 + 24시간)

```
{
  "success": true,
  "data": {
    "slug": "0Wc2yClzAu",
    "url": "http://localhost/h/0Wc2yClzAu",
    "name": "토스 결제 웹훅 테스트",
    "anonymousToken": "wklXhifFLxoLJQcF_LQdBmMckrYMe3zWGhNnwxsOYQA",
    "expiresAt": "2026-10-25T15:11:36.032867597+09:00"
  },
  "error": null
}
```

에러

상태	코드	상황	message
400	INVALID_INPUT	이름이 50자 초과	이름은 50자 이하로 입력해 주세요.
400	INVALID_INPUT	JSON 형식 오류	요청 바디 형식이 올바르지 않습니다.

4. 웹훅 수신

```
ANY /h/{slug}/**
```

외부 서비스(결제사, 깃허브 등)가 호출하는 주소입니다. 인증이 없고, 모든 HTTP 메서드와 하위 경로를 받습니다. `/h/{slug}` , `/h/{slug}/payments/webhook?retry=1` 모두 같은 엔드포인트로 들어갑니다.

저장하는 값: 메서드, 하위 경로(`/payments/webhook`), 쿼리 문자열, 헤더 전체, 바디 원본, Content-Type, 보낸 쪽 IP, 바디 크기(바이트), 받은 시각

- 바디는 폼 파싱 없이 원본 바이트로 읽습니다. 서명 검증처럼 바이트 하나가 중요한 경우를 위해서입니다.
- IP는 Nginx가 넘긴 `X-Forwarded-For` 기준입니다. Nginx 주소가 아니라 실제로 보낸 쪽 주소가 저장됩니다.
- 저장이 끝나면(커밋 후) 이 엔드포인트를 구독 중인 화면에 SSE로 알립니다. 7번 참고.

응답

응답 설정이 없으면 기본 응답을 돌려줍니다.

```
HTTP/1.1 200
Content-Type: application/json

{"ok":true}
```

응답 설정이 있으면 그 상태 코드, 헤더, 바디로 응답하고, `delayMs` 만큼 기다린 뒤 보냅니다. 아래는 `statusCode: 500`, `headers: {"X-Custom": "debug"}`, `delayMs: 3000` 으로 설정한 경우이고, 실제로 3.0초 뒤에 응답이 왔습니다.

```
HTTP/1.1 500
Content-Type: text/plain;charset=UTF-8
X-Custom: debug

{"error":"temporary"}
```

- 헤더에 `Content-Type` 을 넣지 않으면 `text/plain;charset=UTF-8` 로 나갑니다. JSON으로 응답하려면 `Content-Type: application/json` 을 같이 설정해야 합니다.
- `Content-Length`, `Transfer-Encoding`, `Connection`, `Keep-Alive`, `Upgrade`, `TE`, `Trailer` 는 설정해도 무시합니다. 실제 바디와 맞지 않으면 응답이 깨지기 때문입니다.
- 지연 중에는 DB 커넥션을 잡고 있지 않습니다. 로그는 기다리기 전에 이미 저장되어 화면에 보입니다.

에러 (관리 API와 같은 실패 형식)

상태	코드	상황
404	ENDPOINT_NOT_FOUND	없는 slug
410	ENDPOINT_EXPIRED	발급 후 24시간이 지남

5. 요청 목록 조회

```
GET /api/endpoints/{slug}/requests?cursor={requestId}&size={n}
X-Anonymous-Token: {token}
```

받은 요청을 최신순으로 조회합니다. 헤더와 바디는 무거워서 빼고 요약만 내려줍니다.

쿼리 파라미터

이름	타입	필수	기본값	설명
cursor	number	X	(없음)	이전 응답의 `nextCursor`. 없으면 첫 페이지
size	number	X	20	1~100

오프셋(`page=2`) 대신 커서를 쓰는 이유: 보는 도중에 새 웹훅이 들어와도 목록이 밀리지 않습니다. 서버는 `request_id < cursor` 조건으로 조회하고, `size + 1` 개를 가져와서 다음 페이지가 있는지 판단합니다(COUNT 쿼리 없음).

응답 `200 OK`

필드	타입	설명
items[]	array	요청 요약, 최신순
items[].requestId	number	요청 id
items[].method	string	HTTP 메서드
items[].path	string	`/h/{slug}` 뒤의 하위 경로. 없으면 `/`
items[].queryString	string | null	원본 쿼리 문자열 (`?` 제외)
items[].contentType	string | null	요청 Content-Type
items[].ip	string	보낸 쪽 IP
items[].sizeBytes	number	바디 크기(바이트)
items[].receivedAt	string	받은 시각
nextCursor	number | null	다음 페이지 요청에 넣을 값. 마지막 페이지면 null

```
{
  "success": true,
  "data": {
    "items": [
      {
        "requestId": 7,
        "method": "GET",
        "path": "/",
        "queryString": null,
        "contentType": null,
        "ip": "172.21.0.1",
        "sizeBytes": 0,
        "receivedAt": "2026-10-24T15:11:36.139917+09:00"
      },
      {
        "requestId": 6,
        "method": "POST",
        "path": "/payments/webhook",
        "queryString": "retry=1",
        "contentType": "application/json",
        "ip": "172.21.0.1",
        "sizeBytes": 89,
        "receivedAt": "2026-10-24T15:11:36.107941+09:00"
      }
    ],
    "nextCursor": null
  },
  "error": null
}
```


에러: 공통 인증 에러 외에

상태	코드	상황	message
400	INVALID_INPUT	size가 1~100 밖	size는 1~100 사이여야 합니다.
400	INVALID_INPUT	cursor가 숫자가 아님	cursor 값의 형식이 올바르지 않습니다.

6. 요청 상세 조회

```
GET /api/endpoints/{slug}/requests/{requestId}
X-Anonymous-Token: {token}
```

목록에서 뺐던 헤더와 바디까지 모두 내려줍니다.

응답 `200 OK` — 목록의 필드에 아래 두 개가 추가됩니다.

필드	타입	설명
headers	object	`{"이름": ["값", ...]}` . 같은 이름의 헤더가 여러 번 올 수 있어서 값은 배열
body	string | null	바디 원본 문자열. 화면에서 JSON이면 정렬해서 보여 줌


```
{
  "success": true,
  "data": {
    "requestId": 6,
    "method": "POST",
    "path": "/payments/webhook",
    "queryString": "retry=1",
    "headers": {
      "Host": ["localhost"],
      "X-Real-IP": ["172.21.0.1"],
      "User-Agent": ["TossPayments-webhook/1.0"],
      "Content-Type": ["application/json"],
      "Content-Length": ["89"],
      "Accept-Encoding": ["identity"],
      "X-Forwarded-Proto": ["http"]
    },
    "body": "{\"eventType\": \"PAYMENT_STATUS_CHANGED\", \"data\": {\"orderId\": \"ORDER-1\", \"status\": \"DONE\"}}",
    "contentType": "application/json",
    "ip": "172.21.0.1",
    "sizeBytes": 89,
    "receivedAt": "2026-10-24T15:11:36.107941+09:00"
  },
  "error": null
}
```


에러: 공통 인증 에러 외에

상태	코드	상황
404	REQUEST_NOT_FOUND	없는 requestId, 또는 **다른 엔드포인트의 requestId**
400	INVALID_INPUT	requestId가 숫자가 아님

다른 사람의 요청 id를 넣으면 403이 아니라 404입니다. 조회 조건에 `request_id`와 `endpoint_id`를 같이 걸어서 남의 요청은 찾지 못하게 했고, 403을 주면 "그 id가 존재한다"는 정보가 새기 때문에 없는 것과 똑같이 응답합니다.

7. 새 요청 실시간 구독 (SSE)

```
GET /api/endpoints/{slug}/stream
X-Anonymous-Token: {token}
Accept: text/event-stream
```

연결을 열어 두면 이 엔드포인트에 새 웹훅이 들어올 때마다 이벤트가 옵니다. 응답은 `ApiResponse`로 감싸지 않은 `text/event-stream`입니다.

브라우저 `EventSource`는 커스텀 헤더를 보낼 수 없어서, 화면은 `fetch`로 연결하고 `response.body.getReader()`로 스트림을 직접 읽습니다. 토큰을 쿼리 파라미터로 넘기지 않는 이유는 Nginx 접근 로그와 브라우저 기록에 남기 때문입니다.

이벤트

이벤트	data	언제
`connected`	`ok`	연결 직후 한 번
`request`	요청 요약 JSON (5번의 `items[]` 한 개와 같은 모양)	새 웹훅이 저장(커밋)된 뒤
(주석) `: heartbeat`	없음	30초마다. 조용한 연결을 프록시가 끊지 않게

실제 수신 내용:

```
HTTP/1.1 200
Content-Type: text/event-stream

event:connected
data:ok

event:request
data:{"requestId":9,"method":"POST","path":"/push","queryString":null,"contentType":"application/json","ip":"172.21.0.1","sizeBytes":8,"receivedAt":"2026-
```

- 커밋이 끝난 뒤에 보내므로, 알림을 받고 바로 상세를 조회해도 404가 나지 않습니다.
- 연결은 30분 후 서버가 닫습니다. 화면은 끊기면 3초 뒤 다시 연결하고, 그사이 놓친 요청은 목록 API로 다시 받아옵니다. SSE는 "새 요청이 왔다"는 알림이고, 정확한 목록은 목록 API가 기준입니다.
- 에러(401·403·404·410)는 스트림을 열기 전에 검사하므로 일반 JSON 실패 응답으로 옵니다.

8. 응답 설정 조회

```
GET /api/endpoints/{slug}/response
X-Anonymous-Token: {token}
```

웹훅에 돌려줄 응답의 현재 설정을 조회합니다.

응답 `200 OK`

필드	타입	설명
statusCode	number	상태 코드
headers	object	`{"이름": "값"}`
body	string | null	응답 바디
delayMs	number	응답 전 대기 시간(ms)
custom	boolean	true = 사용자가 설정함, false = 아직 설정 전이라 기본 응답 사용 중

설정 전에는 실제로 웹훅에 나가는 기본 응답을 보여 줍니다.

```
{
  "success": true,
  "data": {
    "statusCode": 200,
    "headers": { "Content-Type": "application/json" },
    "body": "{\"ok\":true}",
    "delayMs": 0,
    "custom": false
  },
  "error": null
}
```

에러: 공통 인증 에러

9. 응답 설정 저장

```
PUT /api/endpoints/{slug}/response
X-Anonymous-Token: {token}
Content-Type: application/json
```

설정을 통째로 저장합니다. 없으면 새로 만들고 있으면 덮어씁니다. 같은 요청을 여러 번 보내도 결과가 같습니다(멱등). 저장 직후 들어오는 웹훅부터 바로 적용됩니다.

요청 바디

필드	타입	필수	제약
statusCode	number	O	100~599
headers	object	X	`{"이름": "값"}`, 20개까지. 이름은 HTTP 토큰 문자(영숫자와 `!#$%&'*+.^_`|~-`)만, 값에 줄바꿈(`\r`, `\n`) 불가
body	string	X	20,000자까지
delayMs	number	O	0~5000

```
{
  "statusCode": 500,
  "headers": { "X-Custom": "debug" },
  "body": "{\"error\":\"temporary\"}",
  "delayMs": 3000
}
```

응답 `200 OK` — 저장된 값 (8번과 같은 모양, `custom: true`)

```
{
  "success": true,
  "data": {
    "statusCode": 500,
    "headers": { "X-Custom": "debug" },
    "body": "{\"error\":\"temporary\"}",
    "delayMs": 3000,
    "custom": true
  },
  "error": null
}
```

에러: 공통 인증 에러 외에

상태	코드	상황	message
400	INVALID_INPUT	statusCode 범위 밖, 또는 누락	statusCode는 100~599 사이여야 합니다.
400	INVALID_INPUT	delayMs 범위 밖	delayMs는 0~5000 사이여야 합니다.
400	INVALID_INPUT	헤더 21개 이상	헤더는 20개까지 설정할 수 있습니다.
400	INVALID_INPUT	body 20,000자 초과	바디는 20,000자까지 입력할 수 있습니다.
400	INVALID_INPUT	헤더 이름에 공백 등	헤더 이름에 쓸 수 없는 문자가 있습니다: Bad Name
400	INVALID_INPUT	헤더 값에 줄바꿈	헤더 값에 줄바꿈을 넣을 수 없습니다: X-A
400	INVALID_INPUT	JSON 형식 오류	요청 바디 형식이 올바르지 않습니다.

헤더 값의 줄바꿈을 막는 이유: 여기서 저장한 헤더는 웹훅 응답에 그대로 나갑니다. `"a\r\nSet-Cookie: x"` 같은 값을 허용하면 응답에 헤더를 끼워 넣을 수 있습니다(헤더 주입).

10. 에러 코드

원본은 `backend/src/main/java/com/hooklens/global/exception/ErrorCode.java` 입니다.

코드	상태	기본 message	사용
INVALID_INPUT	400	요청 값이 올바르지 않습니다.	1차
UNAUTHORIZED	401	인증이 필요합니다.	1차
FORBIDDEN_ENDPOINT	403	엔드포인트 소유자가 아닙니다.	1차
ENDPOINT_NOT_FOUND	404	존재하지 않는 엔드포인트입니다.	1차
REQUEST_NOT_FOUND	404	존재하지 않는 요청 로그입니다.	1차
ENDPOINT_EXPIRED	410	만료된 엔드포인트입니다.	1차
INTERNAL_ERROR	500	서버 내부 오류가 발생했습니다.	1차
PAYLOAD_TOO_LARGE	413	요청 바디는 1MB를 넘을 수 없습니다.	2차 (REQ-HOOK-004)
RATE_LIMIT_EXCEEDED	429	요청이 너무 많습니다. 잠시 후 다시 시도해 주세요.	2차 (REQ-HOOK-003)

코드	상태	기본 message	사용
REPLAY_TARGET_BLOCKED	400	내부망 주소로는 재전송할 수 없습니다.	2차 (REQ-RPL-002)
REPLAY_FAILED	502	재전송 대상 서버에 연결하지 못했습니다.	2차 (REQ-RPL-001)

2차 코드는 설계 단계에서 미리 정해 둔 것이고, 아직 응답으로 나가는 곳은 없습니다.